

Tehnici de programare

compilare independentă

Pe măsură ce programele devin din ce în ce mai mari, apare nevoia de a împărți codul în diverse module care să fie mai ușor de gestionat. De exemplu, un program care implementează o bază de date poate fi împărțit în: interfața utilizator (ferestre de introducere date, rapoarte), operarea efectivă asupra bazei de date (adăugare, ștergere, modificare) și operații de intrare/ieșire (salvare/încărcare bază de date, import, export). Unele proiecte pot fi foarte mari. În cazurile extreme, cum este kernelul de Linux, versiunea 5.6 a acestuia are aproximativ 33 milioane de linii de cod, dintre care 60% în diverse drivere.

Pentru gestionarea proiectelor mari, limbajul C pune la dispoziția programatorului mecanismul de **compilare independentă**. Acest mecanism permite următoarele:

- Codul proiectului poate fi împărțit în mai multe fișiere de cod (.c), fiecare fișier implementând un set de funcții sau chiar și numai o singură funcție.
- Fiecare fișier poate avea funcții care să fie vizibile doar în interiorul acelui fișier sau funcții care să fie vizibile global, din toate fișierele proiectului.
- Pentru ca funcțiile și variabilele implementate într-un fișier să fie vizibile și din alte fișiere, se folosesc **declarații** de funcții și de variabile. De obicei aceste *declarații* se grupează în fișiere antet (.h - header)
- Fișierele de cod pot fi compilate independent, separat, doar atunci când este nevoie de compilarea lor. De exemplu, dacă avem un proiect cu 5 fișiere și lucrăm doar la unul dintre ele, doar acesta va fi recompilat la execuția programului. Celelalte fișiere nu mai au nevoie să fie recompilate, deoarece ele nu au fost modificate de la ultima compilare.
- Un proiect mai mare poate conține proiecte mai mici, unele dintre ele fiind aplicații separate (ex: programul de instalare al aplicației), iar altele fiind **biblioteci** (libraries) care se pot folosi în aplicațiile proiectului. O *bibliotecă* nu este o aplicație propriu-zisă ci doar un set de funcții, tipuri de date, variabile, etc., pe care le putem folosi în diverse aplicații.

În continuare, vom discuta folosind un exemplu cum funcționează mecanismul de compilare independentă.

Exemplul 1: Să se implementeze o bază de date care conține persoane, fiecare persoană fiind definită prin nume (max 30 caractere) și salariu de tip *float*. La pornirea aplicației baza de date va fi încărcată automat de pe disc. La ieșirea normală din aplicație (fără eroare), baza de date va scrisă pe disc. Programul va afișa un meniu cu următoarele opțiuni: 1-adăugare, 2-ștergere, 3-listare (în ordine alfabetică), 4-ieșire.

Vom implementa aplicația sub forma unui proiect, denumit "bazadate", format din 6 fișiere (2 antete și 4 de cod), listate mai jos. La începutul fiecărui fișier este numele său și o scurtă descriere.

```
// util.h - functii utilitare de uz general
#pragma once

// afiseaza un text de eroare formatat si iese din program
void err(const char *fmt,...);

// citeste in "dst" o linie de text de maxim "max" caractere
// inainte de a se citi linia, se afiseaza "text"
void citesteText(const char *text,char *dst,int max);
```

```
// citește și returnează o valoare de tip float
// înainte de a se citi valoarea, se afișează "text"
float citeșteFloat(const char *text);
```

```
// util.c - funcții utilitare de uz general
```

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdarg.h>
```

```
void err(const char *fmt,...)
{
    va_list va;
    va_start(va,fmt);
    fprintf(stderr,"eroare: ");
    vfprintf(stderr,fmt,va);
    fputc('\n',stderr);
    va_end(va);
    exit(EXIT_FAILURE);
}
```

```
void citeșteText(const char *text,char *dst,int max)
{
    printf("%s: ",text);
    fgets(dst,max,stdin);
    dst[strcspn(dst,"\r\n")]='\0';    // elimină \n de la sfârșitul liniei
}
```

```
float citeșteFloat(const char *text)
{
    float f;
    printf("%s: ",text);
    scanf("%g",&f);
    return f;
}
```

```
// bd.h - interfata cu baza de date
```

```
#pragma once
```

```
#define MAX_NUME    31
```

```
typedef struct Persoana{
    char nume[MAX_NUME];
    float salariu;
    struct Persoana *urm;
}Persoana;
```

```
extern Persoana *bd;
```

```
// adaugă o nouă persoană la baza de date
```

```
// adaugarea se face in asa fel incat baza de date este mereu sortata dupa nume
```

```
void adauga(const char *nume,float salariu);
```

```
// sterge o persoana din baza de date
```

```
// returneaza 1 daca persoana a fost gasita, altfel 0
```

```
int sterge(const char *nume);
```

```
// elibereaza memoria ocupata de baza de date
```

```
void elibereaza();
```

```
// salveaza baza de date in fisier
```

```
void salveaza();
```

```
// incarca baza de date din fisier
```

```
void incarca();
```

```
// bd.c - functii de operare cu baza de date
```

```
#include "bd.h"
```

```
#include "util.h"
```

```
#include <stdio.h>
```

```
#include <string.h>
```

```
#include <stdlib.h>
```

```
Persoana *bd;
```

```
// creaza un nou element de tip Persoana
```

```
static Persoana *nou(const char *nume,float salariu, Persoana *urm)
```

```
{
    Persoana *p=(Persoana*)malloc(sizeof(Persoana));
    if(!p)err("memorie insuficienta");
    strcpy(p->nume,nume);
    p->salariu=salariu;
    p->urm=urm;
    return p;
}
```

```
void adauga(const char *nume,float salariu)
```

```
{
    Persoana *pred=NULL;           // predecesorul nodului curent
    Persoana *crt;                 // nodul curent
    // itereaza toate numele din bd
    for(crt=bd;crt;pred=crt,crt=crt->urm){
        // daca s-a ajuns la un nume identic cu cel de inserat sau care trebuie sa fie dupa acesta,
        // atunci termina iterarea
        if(strcmp(crt->nume,nume)>=0)break;
    }
    if(pred){                      // inserare in interiorul sau la sfarsitul listei
        pred->urm=nou(nume,salariu,crt);
    }else{                         // inserare la inceputul listei
        bd=nou(nume,salariu,crt);
    }
}
```

```

}

int sterge(const char *nume)
{
    Persoana *pred=NULL;           // predecesorul nodului curent
    Persoana *crt;                  // nodul curent
    for(crt=bd;crt;pred=crt,crt=crt->urm){
        if(!strcmp(crt->nume,nume)){
            if(pred){                // stergere din interiorul sau de la sfarsitul listei
                pred->urm=crt->urm;
            }else{                    // stergere de la inceputul listei
                bd=crt->urm;
            }
            free(crt);
            return 1;
        }
    }
    return 0;
}

void elibereaza()
{
    Persoana *crt,*urm;
    for(crt=bd;crt;crt=urm){
        urm=crt->urm;
        free(crt);
    }
    bd=NULL;                        // pentru a se putea refolosi
}

```

// fis.c - functii de salvare si incarcare din fisier a bazei de date

```

#include "bd.h"
#include "util.h"

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

void salveaza()
{
    FILE *fis=fopen("bd.txt","wb");
    if(!fis)err("nu se poate crea bd.txt");
    Persoana *p;
    for(p=bd;p;p=p->urm){
        // fiecare persoana este salvata in fisier pe cate o linie,
        // cu ',' ca separator intre nume si salariu
        fprintf(fis,"%s;%g\n",p->nume,p->salariu);
    }
    fclose(fis);
}

void incarca()

```

```

{
    elibereaza();
    FILE *fis=fopen("bd.txt", "rb");
    // daca nu s-a putut deschide fisierul nu este eroare,
    // deoarece este posibil sa fie prima rulare a programului
    if(!fis)return;
    char linie[MAX_NUME+32]; // o dimensiune acoperitoare pentru o linie din fisier (nume+';'+salariu+\n)
    // citeste linie cu linie
    while(fgets(linie, MAX_NUME+32, fis)){
        char *sep=strchr(linie, ';'); // cauta separatorul dintre nume si salariu
        if(!sep)continue; // daca o linie nu contine separatorul, nu o ia in considerare
        int nNume=sep-linie; // numarul de caractere din nume
        if(nNume>=MAX_NUME)nNume=MAX_NUME-1; // ne asiguram ca numele citit nu este prea mare
        char nume[MAX_NUME];
        memcpy(nume, linie, nNume);
        nume[nNume]='\0'; // adauga separator
        float salariu=(float)atof(sep+1);
        adauga(nume, salariu);
    }
    fclose(fis);
}

```

// main.c - meniu utilizator si comportament global

```

#include "util.h"
#include "bd.h"

#include <stdio.h>

int main()
{
    char nume[MAX_NUME];
    float salariu;
    Persoana *p;

    incarca();
    for(;;){
        printf("1. adaugare\n");
        printf("2. stergere\n");
        printf("3. listare\n");
        printf("4. iesire\n");
        int op; // optiunea
        printf("optiune: ");scanf("%d",&op);
        switch(op){
            case 1:
                getchar();
                citesteText("nume", nume, MAX_NUME);
                salariu=citesteFloat("salariu");
                adauga(nume, salariu);
                break;
            case 2:
                getchar();
                citesteText("nume", nume, MAX_NUME);

```

```

    if(sterge(ume)){
        printf("%s a fost sters din baza de date\n",ume);
    }else{
        printf("%s nu exista in baza de date\n",ume);
    }
    break;
case 3:
    for(p=bd;p=p->urm){
        printf("%s\t%g\n",p->ume,p->salariu);
    }
    break;
case 4:
    salveaza();
    elibereaza();
    return 0;
default:printf("optiune invalida\n");
}
}
}

```

Declarații și definiții

Un **simbol** este nume definit de programator, de exemplu o funcție, variabilă, nume de tip, structură, etc. În C, **declarația** și **definiția** unui simbol sunt termeni diferiți:

- **declarația** - este specificarea tipului unui simbol, fără a se furniza o *implementare* (sau *corp*) pentru acel simbol. Diverse simboluri se declară în felul următor:
 - *funcții* - se pune ; (punct și virgulă) după lista de parametri ai funcției (ex: *int max(int a,int b);*)
 - *structuri* - se pune ; (punct și virgulă) după numele structurii (ex: *struct Punct;*)
 - *variabile* - se pune **extern** în fața variabilei sau, pentru vectori, nu se mai trece numărul de elemente (ex: *extern int a; float v[];*)
- **definiția** - este implementarea propriu-zisă a unui simbol, de exemplu o structură sau funcție cu tot cu acoladele corpului ei, o declarație de vector cu dimensiune, etc.

Înainte de a folosi un simbol, compilatorul de C trebuie să știe tipul celui simbol. De exemplu, nu se poate folosi o funcție dacă nu avem înainte de folosirea ei o *declarație* sau *definiție* pentru ea. Din acest motiv, dacă avem o funcție definită într-un fișier .c și dorim să o folosim într-un al doilea fișier .c, trebuie prima oară să o *declaram* în al doilea fișier, pentru ca compilatorul să știe despre ce este vorba. Pentru aceasta folosim fișiere antet (.h - header). Aceste fișiere conțin doar *declarații* de simboluri, la care se pot adăuga și *definiții* de structuri. Așa cum am discutat la preprocesorul de C, directiva *#include* se substituie direct cu corpul fișierului specificat, ca și cum am face copy/paste cu conținutul celui fișier. Astfel, dacă într-un fișier antet punem o declarație de funcție și includem acel antet într-un alt fișier .c, este ca și cum am scrie declarația de funcție direct în fișierul .c. În acest mod putem specifica faptul că în proiect există o funcție cu numele și tipul declarat, dar care se află implementată (*definită*) în alt fișier.

În exemplul de mai sus, funcția *err* este implementată în fișierul *util.c*. Pentru a folosi această funcție și în alte fișiere, am pus *declarația* ei în antetul *util.h* și am inclus acest antet în toate fișierele care folosesc *err*. Astfel, ea a devenit vizibilă și în alte fișiere.

Dacă avem nevoie ca unele variabile să fie vizibile din mai multe fișiere de cod, putem să le declarăm în antet punând în fața lor cuvântul cheie **extern**. Acesta înseamnă că avem de-a face doar cu *declarația* variabilei, nu cu *definiția* ei. În exemplul nostru, variabila *bd* este folosită în mai multe fișiere, astfel încât a trebuit să o facem vizibilă pentru toate aceste fișiere cu declarația "*extern Persoana *bd;*" (în antetul *bd.h*). Dacă nu am fi pus *extern* în față, în fișierele de cod ar fi apărut "*Persoana *bd;*", ceea ce înseamnă de fapt *definiția* variabilei. În acest caz,

compilatorul ar fi dat o eroare de redefinire, fiindcă ar fi constatat că variabila *bd* este definită în mai multe fișiere de cod.

Pentru variabile de tip vector se preferă ca declararea lor să se facă prin omiterea numărului de elemente din vector. De exemplu, *"int v[];"* este considerată ca fiind o *declarație*, nu o *definiție*, deoarece lipsește numărul elementelor.

Organizarea fișierelor antet și a celor de cod

Fișierele antet specifică **interfața** unui anumit modul. Tot ceea ce într-un anumit modul este destinat folosinței publice (utilizare în alte fișiere) se va *declara* și în fișierul antet pentru acel modul. Dacă o anumită funcție sau variabilă nu face parte din interfață (ex: ea este folosită doar pentru implementarea internă a modulului), declarația acelei funcții nu se va regăsi în antet. Mai mult decât atât, limbajul C permite ca la implementarea unei funcții să punem în față cuvântul cheie **static**, ceea ce înseamnă că funcția respectivă se va folosi strict doar în cadrul acelui fișier și nu este vizibilă în alt fișier. De exemplu, funcția *nou* din *bd.c* a fost considerată ca fiind un auxiliar intern pentru implementarea bazei de date și este folosită doar în acest fișier. Din acest motiv, a fost definită ca fiind *static*. Numele funcțiilor statice se pot repeta și în alte module, fără a fi considerate erori de redefinire (ex: putem avea o altă funcție numită *nou* în fișierul *main.c*).

Nu există o corespondență de unu la unu între un antet și un fișier de cod. Putem avea un antet a cărui funcții să fie implementate într-un singur fișier de cod (ex: toate funcțiile declarate în *util.h* au fost implementate în *util.c*) sau putem să avem funcțiile dintr-un antet implementate în mai multe fișiere de cod (ex: funcțiile declarate în *bd.h* au fost implementate în fișierele *bd.c* și *fis.c*). Putem să avem chiar antete la care nu le corespunde niciun fișier de cod (ex: un antet care definește doar tipuri de date: structuri, nume de tipuri, enumerări).

Criteriile de împărțire pe fișiere sunt date mai mult de legătura logică care formează un anumit set de funcții și de mărimea implementării lor. De exemplu, funcțiile din *util.h* sunt funcții utilitare de uz general, care pot fi refolosite fără nicio modificare și în alte proiecte. Dacă analizăm funcția *err*, constatăm că ea nu are niciun fel de legătură cu baza de date. Funcțiile din *bd.h* implementează baza de date, dar am considerat că este prea mult cod de scris într-un singur fișier, așa că l-am împărțit în două fișiere, folosind un subcriteriu de organizare: dacă sunt funcții legate de fișiere sau nu.

În general este bine ca modulele să fie cât mai independente unele de altele (cât mai decuplate). În acest fel, pe de o parte putem să le refolosim ușor și în alte aplicații, iar pe de altă parte, dacă modificăm ceva într-un modul, nu va trebui să facem multe modificări și în alte module, dacă nu se schimbă interfața dintre ele. În exemplul nostru, toate funcțiile de operare asupra bazei de date (interfața definită de *bd.h*) sunt complet *agnostice* (neutre, independente) în privința modulului de introducere sau de afișare a datelor. Nu vom găsi în *bd.c* sau în *fis.c* nicio folosire de *printf*, *scanf*, *fgets*, etc, decât eventual la apariția unei erori (prin funcția *err*). Din acest motiv, toată implementarea bazei de date se poate folosi practic nemodificată și dacă vom dori ca aplicația noastră să aibă o interfață utilizator grafică. Pentru aceasta vor trebui să fie modificate doar funcțiile din *util* și din *main*.

Când avem în față un program făcut de altcineva, este bine să începem analiza sa cu fișierele antet. În acestea vom găsi interfața de comunicare dintre module și totodată tipurile de date cele mai importante, care se folosesc în mai multe module. După ce ne-am familiarizat cu interfața, putem pe urmă să inspectăm și fișierele de cod (.c) pentru a vedea cum anume s-au implementat diverse funcții. Din acest motiv, este bine să punem comentariile care explică funcțiile în fișierele antet, fiindcă acolo ne vom uita prima oară să vedem cum se folosesc acele funcții.

Includerea antetelor în fișierele de cod

Când includem antete în fișierele de cod, este bine ca prima oară să includem antetele aplicației respective (între ghilimele, "...", ceea ce înseamnă că prima oară se caută aceste fișiere în directoarele aplicației) și abia apoi antetele standard (între <...>, ceea ce înseamnă că antetele se caută prima oară în directoarele bibliotecii standard C). Prin faptul că includem prima oară antetele aplicației, ne asigurăm de faptul că ele sunt complete, adică din ele nu lipsesc declarațiile unor tipuri folosite în antet.

De exemplu, să presupunem că un fișier antet al nostru folosește la o declarație tipul de date `size_t` și acest antet nu mai include la rândul lui alte fișiere. În acest caz am avea o eroare, deoarece tipul `size_t` este definit în antetele standard C (mai exact în `stddef.h`), pe care antetul nostru nu le include. Este posibil ca în anumite fișiere sursă să putem folosi acest antet (de exemplu, dacă este precedat de `string.h`, care include ceea ce este necesar pentru `size_t`), iar în alte fișiere sursă să avem o eroare de genul "`size_t` este nedefinit". Dacă vom include prima oară antetul nostru, deoarece înainte de el nu este niciun alt antet care să definească `size_t`, vom primi imediat mesajul de eroare și vom putea astfel să remediem eroarea. Un fișier antet poate la rândul lui să includă alte fișiere antet, deci, în acest caz, la începutul antetului va trebui să includem `stddef.h`, în care este definit `size_t`.

Asigurarea unicității includerii fișierelor antet

În proiecte pot exista situații mai complexe, în care mai multe fișiere antet să includă fiecare același antet. Dacă vom include aceste fișiere în cod, antetul comun va fi inclus de mai multe ori, ceea ce poate duce la anumite erori și la creșterea timpului de compilare. De exemplu, dacă antetul comun definește o structură, acea definiție va fi inclusă de mai multe ori și se va genera o eroare de genul "redefinire structură". Din aceste motive, este bine să ne asigurăm că un antet va fi inclus o singură dată într-un fișier de cod (dar evident că el se poate include în oricâte fișiere de cod).

Pentru a se asigura unicitatea includerii unui antet, se folosește directiva **#pragma once**, pusă chiar la începutul antetului respectiv. Această directivă lasă să fie inclus antetul la prima incluziune, dar, dacă ulterior se va încerca din nou includerea sa în același fișier de cod (chiar dacă această incluziune se face prin intermediul altor antete), nu se va mai include nimic din acel antet.

#pragma once nu a existat în primele versiuni ale limbajului C, astfel încât inițial a fost folosită o altă modalitate de asigurare a unicității includerii, modalitate care este redată în codul de mai jos și pe care este posibil să o găsiți în multe antete:

```
#ifndef util_h
#define util_h
//... corp antet ...
#endif
```

Cu această metodă, la prima incluziune a unui antet definește un simbol, pe cât posibil unic pentru acel antet (de exemplu numele lui). Se poate constata că corpul antetului este inclus doar dacă acest simbol unic nu este definit, deci antetul încă nu a mai fost inclus în acel fișier de cod. Dacă se constată că simbolul este deja definit, deci antetul a fost deja inclus, atunci nu se mai compilează nimic din corpul antetului.

Compilarea aplicațiilor formate din mai multe fișiere de cod

Când un proiect C este compilat la cod mașină, de fapt au loc două faze distincte:

1. *Compilarea fiecărui fișier de cod la **cod obiect*** - în această fază, din fiecare fișier `.c` rezultă un **fișier obiect** (object, cu extensia `.o` în Linux sau `.obj` în Windows). Un *fișier obiect* conține codul mașină pentru funcțiile din acel fișier, plus o serie de informații suplimentare, de exemplu numele funcțiilor, variabilelor, etc din fișier.
2. *Legarea (link - asamblarea, linkeditarea) codurilor obiect împreună* - în această fază se preia doar codul util (cel al funcțiilor folosite de aplicație) din fiecare fișier obiect. Toate aceste coduri se leagă împreună (link) și astfel rezultă aplicația finală (fișierul executabil).

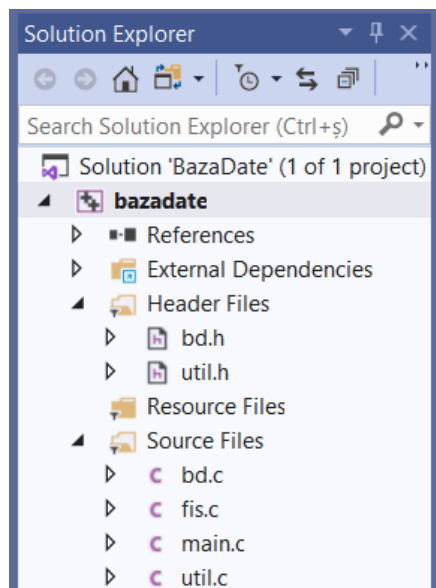
Până acum, când am folosit în linia de comandă `gcc`, aceste faze au avut loc automat, rezultând direct aplicația finală. Dacă vrem să compilăm la cod obiect un fișier `.c`, vom folosi în linia de comandă argumentul **"-c"** (compile only), ceea ce are ca efect doar generarea codului obiect pentru fișierul respectiv. Nu contează ordinea în care compilăm fișierele de cod. În final, după ce avem toate fișierele obiect, legarea lor (link) se face tot cu `gcc`, înșirând în linia de comandă toate fișierele obiect pe care dorim să le legăm împreună. Nu contează ordinea în care sunt înșiruite fișierele obiect. Pentru exemplul de mai sus, secvența de operații este:


```
gcc -Wall -c util.c
gcc -Wall -c bd.c
gcc -Wall -c fis.c
gcc -Wall -c main.c
gcc -Wall -o bazadate util.o bd.o fis.o main.o
```

Această secvență de operații va genera aplicația executabilă (*bazadate* în Linux sau *bazadate.exe* în Windows).

Există și unele programe utilitare care automatizează procesul de compilare al proiectelor. Printre acestea sunt *cmake*, *make*, etc. Pentru folosirea acestora, se scriu niște fișiere care cuprind regulile de compilare (ex: aplicația *bazadate* este compusă din codul obiect rezultat din compilarea fișierelor *util.c*, *bd.c*, *fis.c*, *main.c*). Utilitarele respective vor invoca *gcc*-ul (și alte programe), vor compila doar fișierele modificate între timp, vor ține cont dacă un fișier s-a compilat cu succes sau nu, etc.

Toate IDE-urile moderne pot lucra cu proiecte. Unele IDE-uri pot lucra chiar cu proiecte ce conțin subproiecte. O *soluție* (solution) în *Visual Studio* poate conține oricâte proiecte. De exemplu, un proiect poate fi o bibliotecă de funcții, iar altul aplicația principală, care utilizează acea bibliotecă. Exemplul de mai sus poate arăta în Visual Studio în felul următor:



IDE-urile pot dispune de algoritmi complecși de compilare independentă. De exemplu, ele pot analiza fișierele *.c* să vadă fiecare fișier ce antete include și vor recompila acel fișier *.c* doar dacă s-a modificat între timp unul dintre antetele incluse de el. În același timp, dacă se dispune de un CPU cu mai multe *nuclee* (cores), se pot compila simultan mai multe fișiere *.c*, pentru a se utiliza toate nucleele disponibile. Aceste mecanisme pot salva mult timp de așteptare la compilarea proiectelor mari.

Biblioteci (libraries)

Codul C poate fi compilat pentru a rezulta o *bibliotecă de funcții* (nu o aplicație executabilă). O bibliotecă este formată din mai multe fișiere obiect arhivate împreună. În acest fel putem avea la dispoziție funcții (uneori foarte complexe), pe care nu mai trebuie să le scriem noi și pe care le putem include în aplicațiile noastre. Inclusiv funcțiile standard C (ex: *printf*) sunt puse la dispoziția programatorului tot sub forma unei biblioteci de funcții.

Există foarte multe biblioteci, pentru cele mai variate scopuri: lucrul cu imagini bitmap (libjpeg, libpng), arhive (zlib), grafică 3D (OpenGL, Vulkan, DirectX), baze de date SQL (sqlite), interfețe utilizator grafice (gtk), folosirea protocoalelor de rețea (libcurl), etc. Folosirea unor biblioteci existente poate duce la economii substanțiale de muncă. De exemplu, la unele dintre aceste biblioteci au lucrat zeci sau chiar sute de programatori, de-a lungul a mai multor ani. Când începem lucrul la o nouă aplicație, putem într-o primă fază să identificăm ce gen de

funcționalități sunt necesare, iar apoi să căutăm dacă găsim bibliotecile potrivite sau dacă putem adapta un cod deja existent.

Aplicații propuse

Aplicația 11.1: La exemplul din laborator să se adauge opțiunea de meniu “listeaza după salariu”, care va lista persoanele în ordinea crescătoare a salariului.

Aplicația 11.2: La exemplul din laborator să se adauge opțiunea de meniu “modifică”. Pentru aceasta se cere numele persoanei a cărei înregistrare va fi modificată, iar apoi se vor cere noul nume și salariu. Baza de date trebuie să rămână sortată alfabetic, chiar dacă noul nume diferă de cel anterior.

Aplicația 11.3: La exemplul din laborator să se adauge câmpul “sex” de tip *char* la structura *Persoana*, cu două valori: ‘m’ - masculin, ‘f’ - feminin. Se vor modifica funcțiile de adăugare, salvare, încărcare și listare pentru a opera și cu acest câmp.

Aplicația 11.4: Să se implementeze un modul care definește următoarele funcții pe șiruri de caractere: *strCat(s1,s2)* - concatenează șirurile *s1* și *s2* într-o zonă alocată dinamic și o returnează; *strExtract(begin,end)* - copiază șirul de caractere care începe de la pointerul *begin* și se termină la pointerul *end* (exclusiv) într-o zonă alocată dinamic și o returnează; *strTrim(s)* - elimină caracterele *\n\r\t* și spațiu de la începutul și de la sfârșitul șirului *s* și returnează noul șir într-o zonă alocată dinamic. Să se scrie o aplicație care testează funcțiile din acest modul, folosind șiruri de caractere introduse de la tastatură.

Aplicația 11.5: Să se scrie o aplicație care implementează tipul de date *VecInt*, un vector redimensionabil automat cu elemente de tip *int*. Acest tip se va implementa într-un fișier .c separat, având ca interfață funcții pentru: crearea unui vector nou, adăugarea unui element la vector, ștergerea unui vector, returnarea elementului de la o poziție dată, setarea elementului de la o poziție dată, returnarea numărului de elemente din vector. Aplicația va citi de la tastatură valori într-un *VecInt* până la apariția valorii 0, exclusiv, le va sorta și va afișa valorile sortate.